



哈爾濱工業大學

HARBIN INSTITUTE OF TECHNOLOGY



网络通信基础(II)

IP寻址、DNS与Socket编程

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 网络层与IP协议**
- 2 应用层与DNS协议**
- 3 Socket编程**

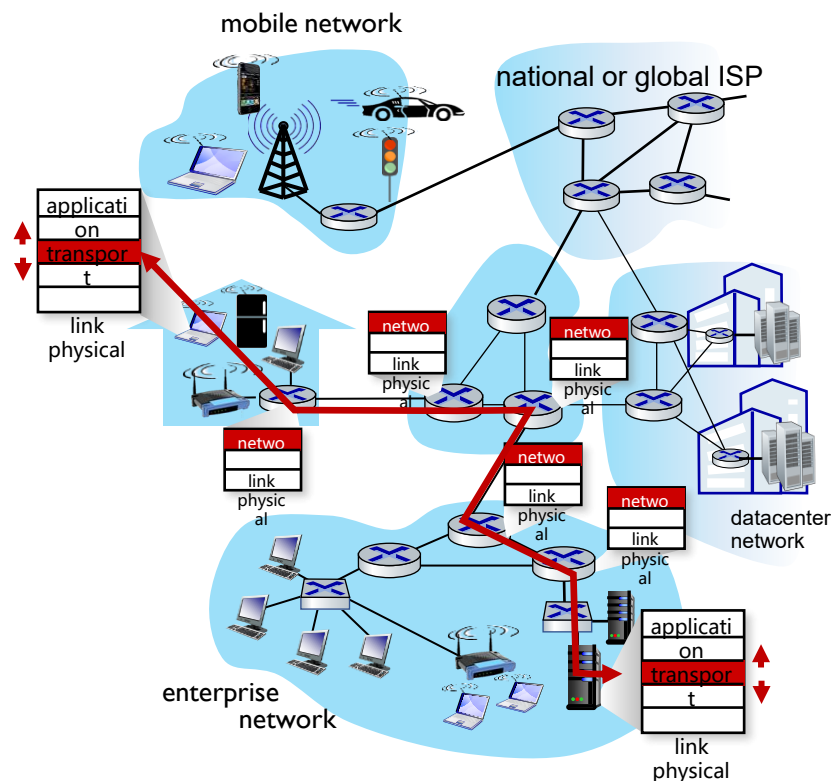
目录

Contents

- 1 网络层与IP协议**
- 2 应用层与DNS协议
- 3 Socket编程

网络层服务和协议

- 从发送到接收主机的传输段
 - sender: 将片段封装为数据报, 传递到链接层
 - receiver: 将段传送到传输层协议
- 每个互联网设备中的网络层协议: 主机、路由器
- routers:
 - 检查所有通过它的 IP 数据报中的标头字段
 - 将数据报从输入端口移动到输出端口, 以沿端路径传输数据报



Network Layer: 4-4

两个关键的网络层功能

网络层函数:

- forwarding: 将数据包从路由器的输入链路移动到适当的路由器输出链路
- routing: 确定数据包从源到目的地的路由
- 路由算法

类比: 进行一次旅行

- forwarding: 通过单立交桥的过程
- routing: 从始发地到目的地的计划之旅的过程



forwarding

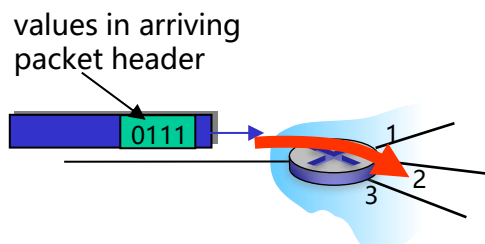


routing

网络层:数据平面、控制平面

数据平面:

- 本地,每个路由器函数
- 确定到达路由器输入端口的数据报如何转发到路由器输出端口

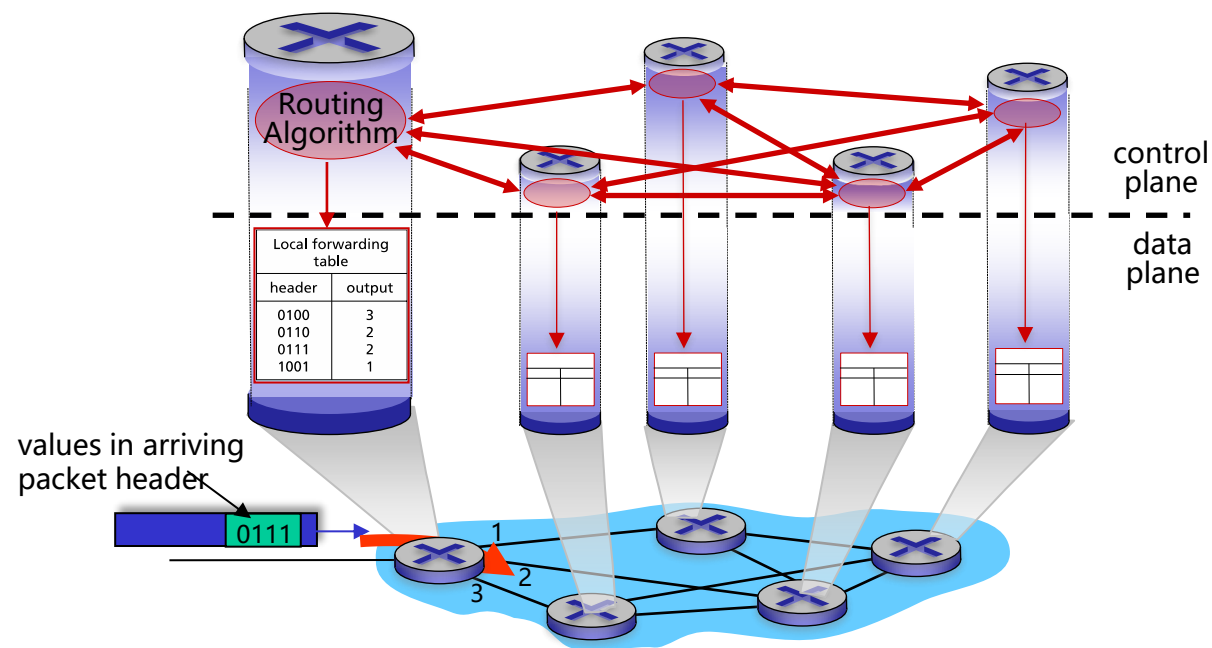


控制平面

- 全网逻辑
- 确定数据报在路由器之间如何沿从源主机到目标主机的端端路径路由
- 两个控制平面方法:
 - 传统的路由算法: 在路由器中实现
 - 软件定义的网络 (SDN) : 在 (远程) 服务器中实现

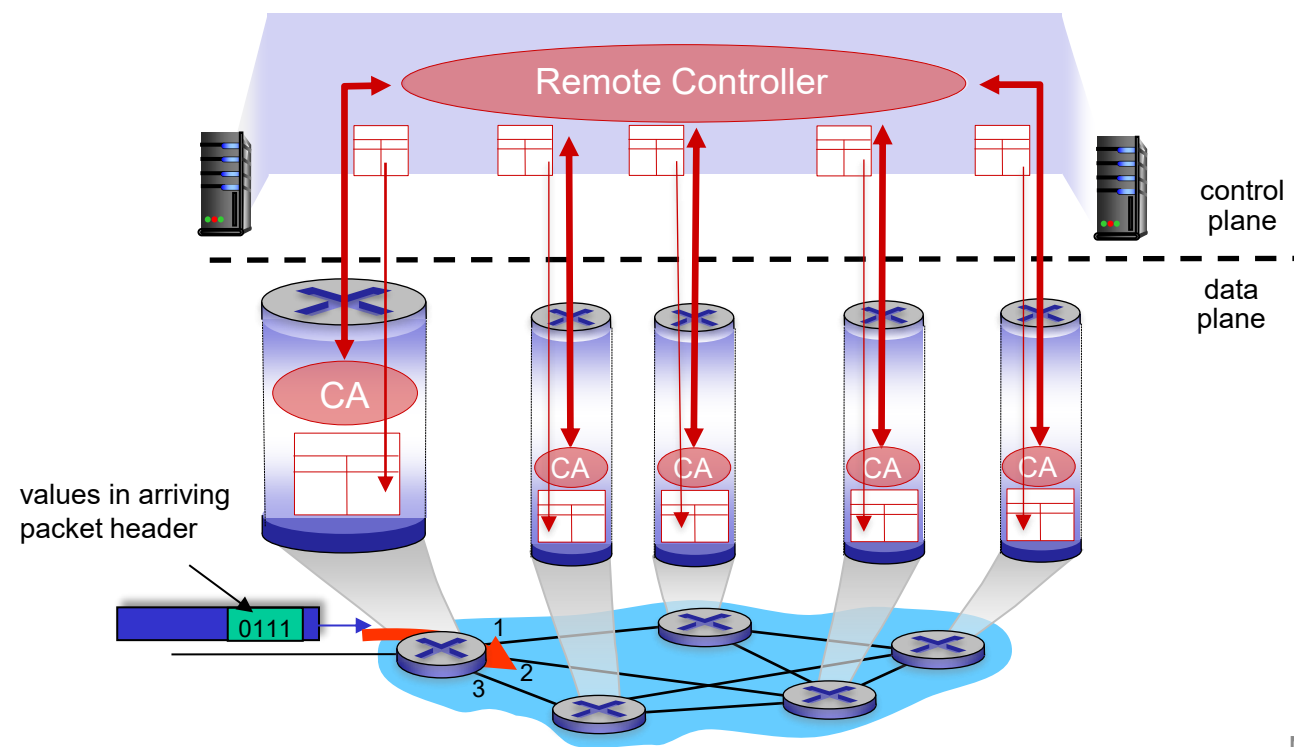
每路由器控制平面

每个路由器中的单个路由算法组件在控制平面中交互



软件定义网络 (SDN) 控制平面

遥控器计算,在路由器中安装转发表



Network Layer: 4-8

Q: "通道"将数据报从发送方传输到接收方的服务模式是什么?

单个数据报的示例服务:

- 保证交付
- 保证交付,延迟少于 40 毫秒

数据报流的示例服务:

- 按顺序数据报交付
- 保证流的最小带宽
- 包间间距变化的限制

网络层服务模型

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no

互联网"尽力而为"服务模式

无保证:

- i. 成功将数据报传送到目的地
- ii. 交货时间或顺序
- iii. 端端流可用的带宽

网络层服务模型

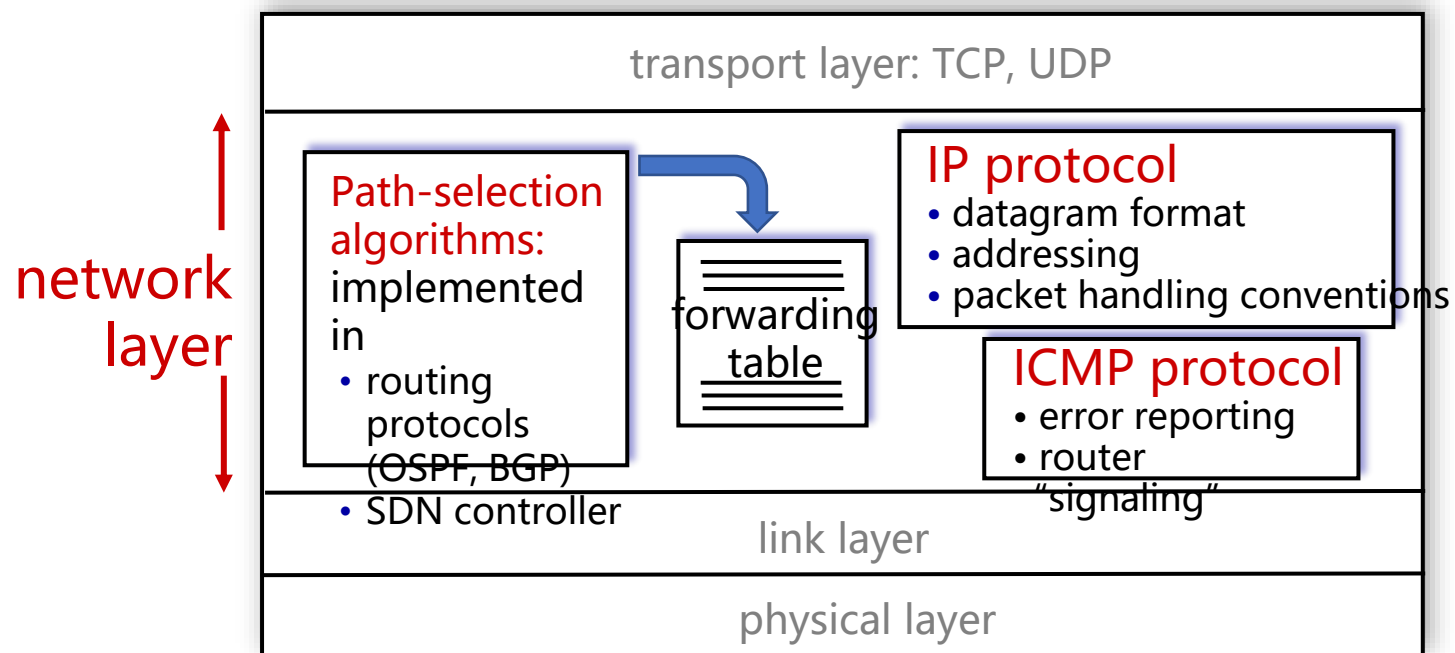
Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	Constant Bit Rate	Constant rate	yes	yes	yes
ATM	Available Bit Rate	Guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

尽力服务的思考:

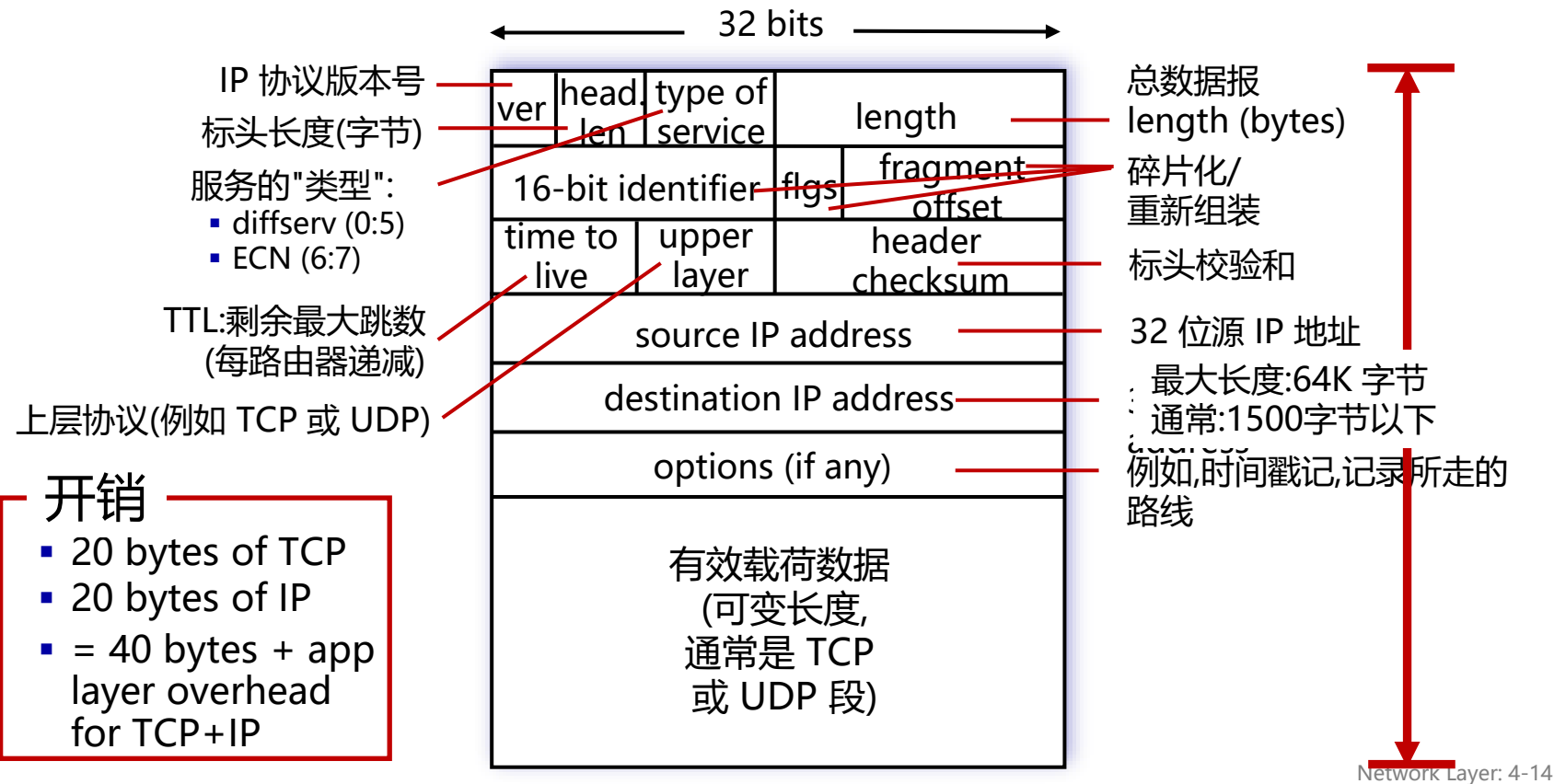
- 机制简单,使互联网得以广泛采用
- 带宽充足,实时应用程序(例如交互式语音、视频)的性能在"大部分时间"中"足够好"
- 复制的应用层分布式服务(数据中心、内容分发网络)与客户网络相连,允许从多个位置提供服务
- "弹性"服务的拥塞控制有助于

尽力服务模式的成功是难以争辩的

主机、路由器网络层功能:

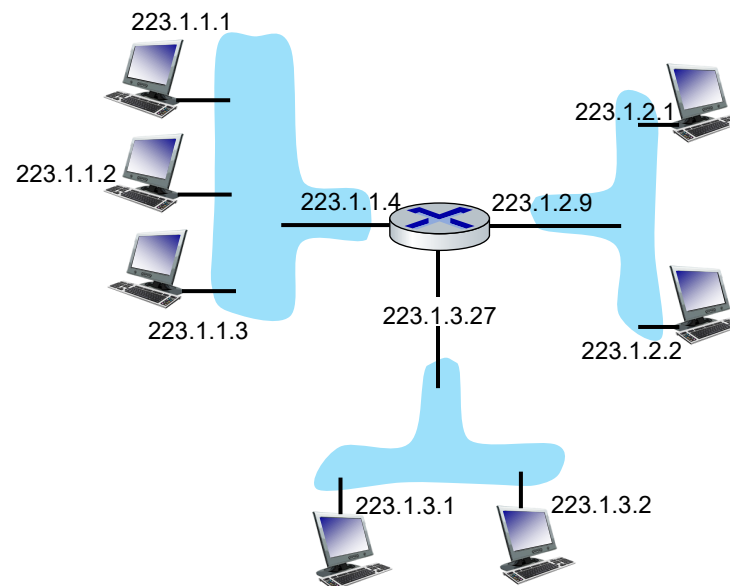


IP Datagram 格式



IP 寻址:简介

- IP地址:与每个主机或路由器接口关联的 32 位标识符
- 接口:主机/路由器与物理链路之间的连接
- 路由器通常有多个接口
- 主机通常有一个或两个接口(例如,有线以太网、无线 802.11)



点十进制 IP 地址表示法:

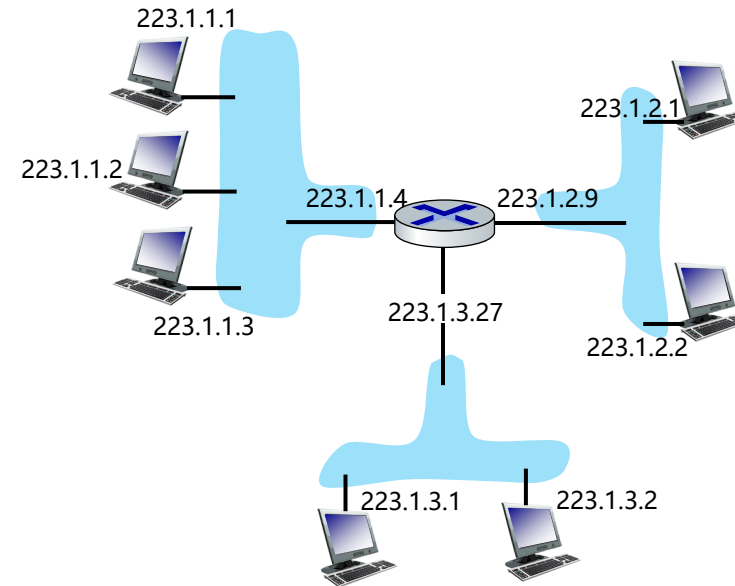
223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

Network Layer: 4-15

IP 寻址:简介

- **IP address:** 32-bit identifier associated with each host or router **interface**
- **interface:** connection between host/router and physical link
 - router's typically have multiple interfaces
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)



dotted-decimal IP address notation:

223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

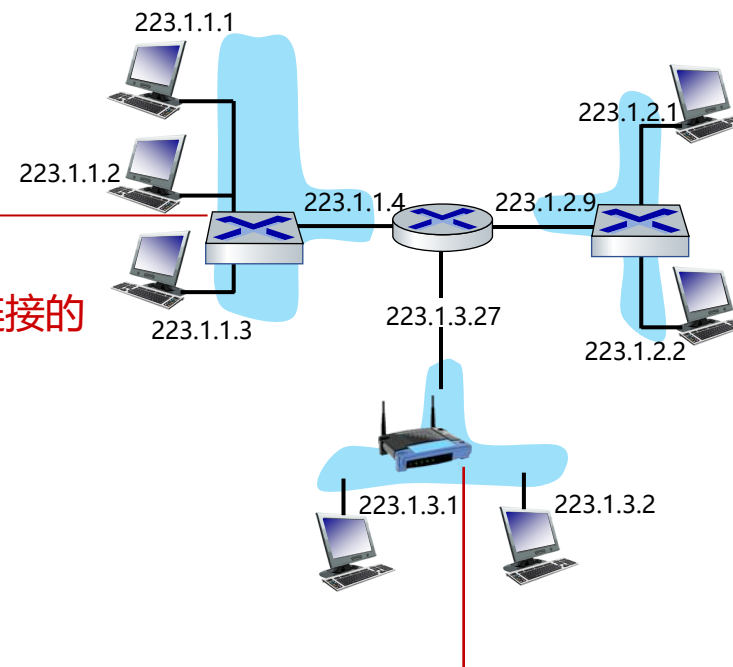
Network Layer: 4-16

IP 寻址:简介

Q:接口如何实际连接?

A:有线
以太网交换机连接的
以太网接口

目前:无需担心一个接口如何连接到
另一个接口(没有中间路由器)



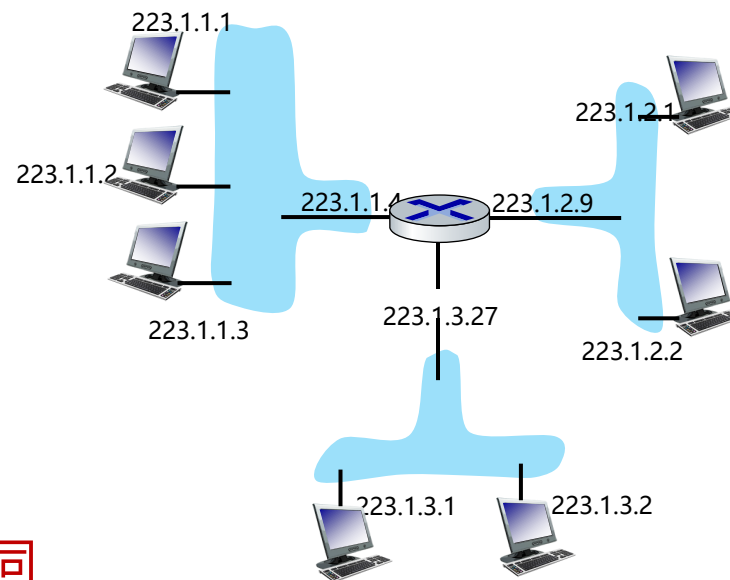
A:无线 WiFi 接口由 WiFi 基站连接

■ 什么是子网?

- 无需经过中间路由器即可物理到达的设备接口

■ IP 地址有结构:

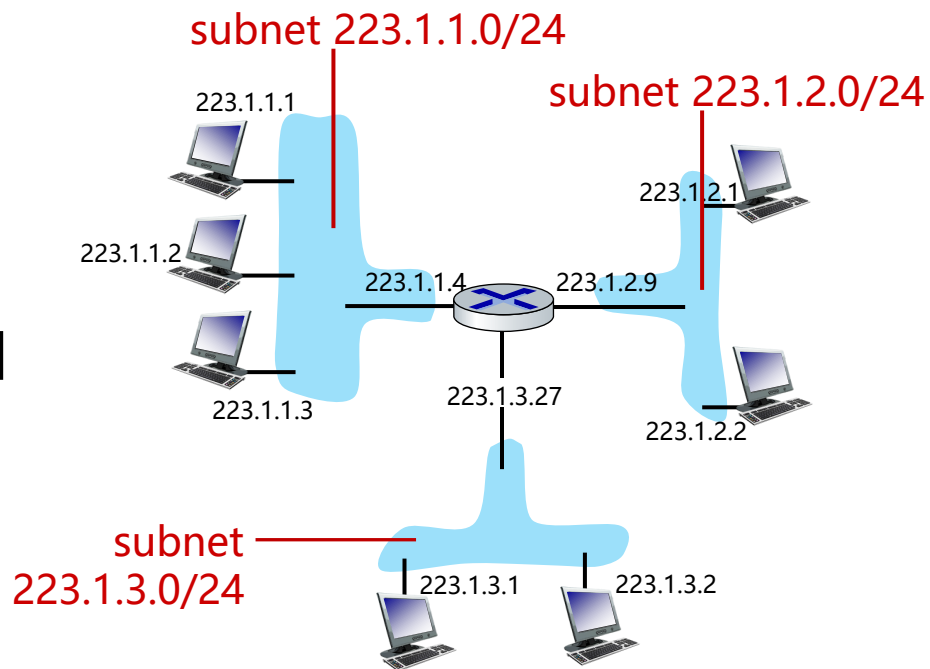
- 子网部分:同一子网中的设备具有共同的高阶位
- 主机部分:其余低阶位



由 3 个子网组成的网络

定义子网的配方:

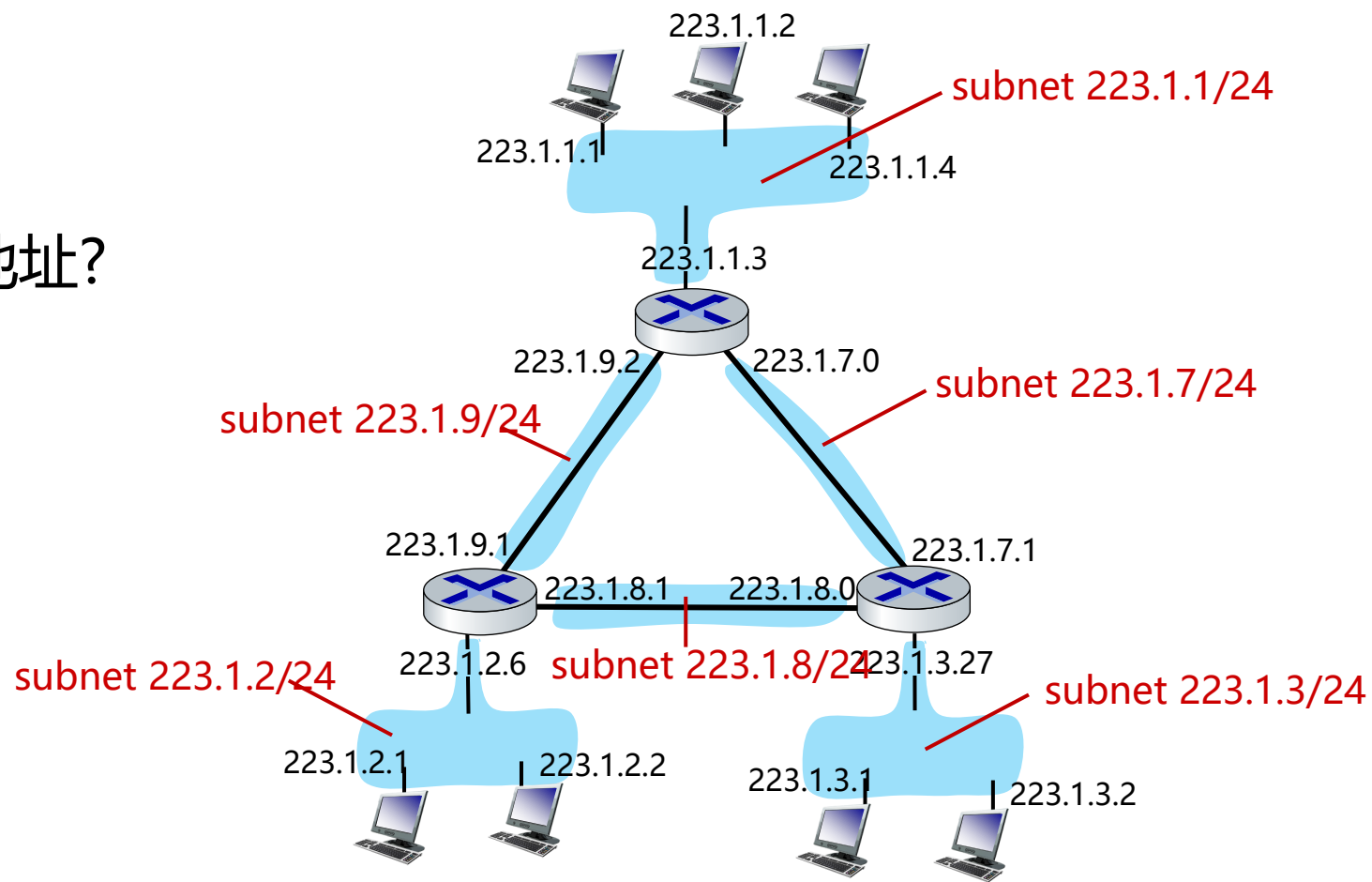
- 将每个接口与其主机或路由器分离,形成孤立网络的"孤岛"
- 每个孤立的网络称为子网



子网掩码:/24
(高阶 24 位:IP 地址的子网部分)

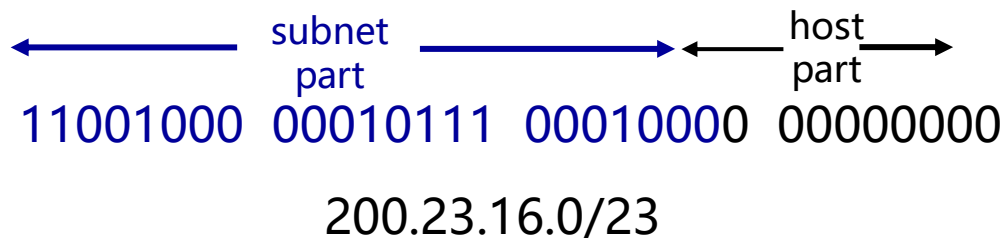
子网

- 子网在哪里?
- 什么是 /24 子网地址?



CIDR: Classless InterDomain Routing (发音 “cider”)

- 任意长度地址的子网部分
- 地址格式:A.B.C.D/X,其中 x 是地址子网部分的 # 位



IP 地址:如何获得?

这其实是两个问题:

- 1.Q:主机如何获取其网络内的 IP 地址(地址的主机部分)?
- 2.Q:网络如何为自己获取 IP 地址(地址的网络部分)

主机如何获取 IP 地址?

- 由 sysadmin 硬编码在配置文件中(例如 UNIX 中的 /etc/rc.config)
- **DHCP**: **D**ynamic **H**ost **C**onfiguration **P**rotocol:从 AS 服务器动态获取地址
- "即插即用"

DHCP:动态主机配置协议

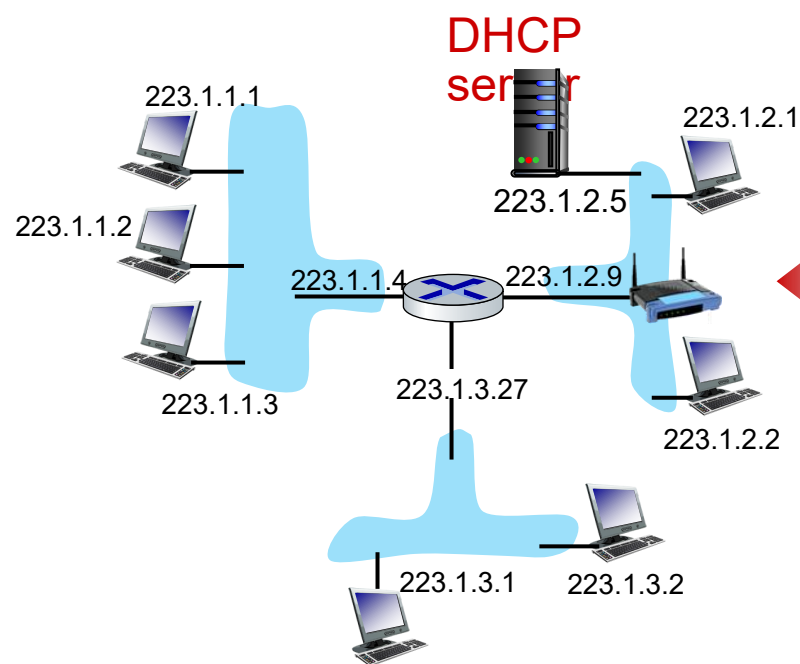
目标:主机在"加入"网络时从网络服务器动态获取 IP 地址

- 可以在使用地址上续租
- 允许重复使用地址(仅在连接/打开时保留地址)
- 支持加入/离开网络的移动用户

DHCP概述:

- host broadcasts DHCP discover msg [可选]
- DHCP 服务器以 DHCP offer msg 响应 [可选]
- 主机请求 IP 地址:DHCP 请求消息
- DHCP 服务器发送地址:DHCP ack msg

DHCP 客户端-服务器场景

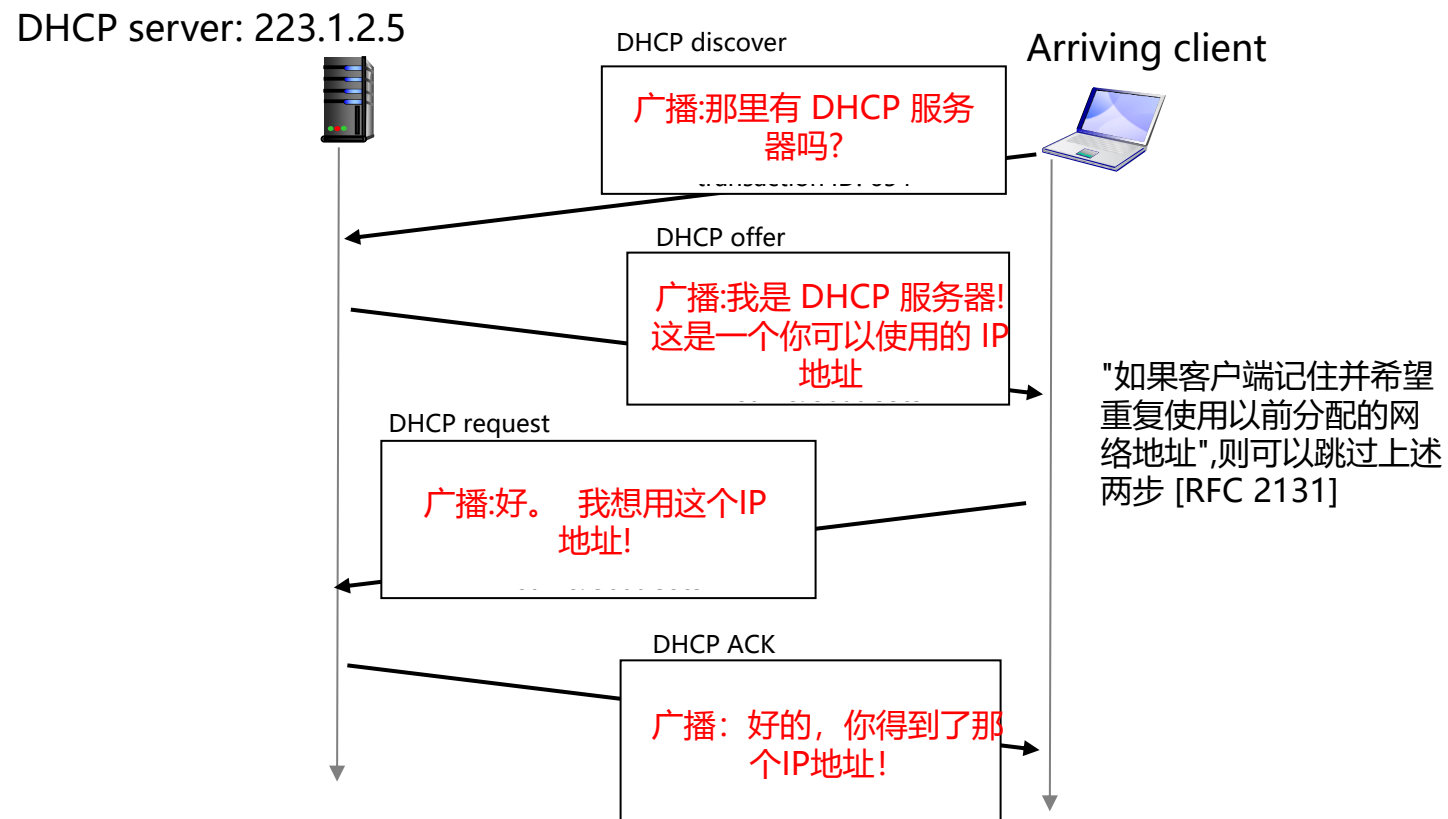


通常,DHCP 服务器将位于路由器中,为路由器所连接的所有子网提供服务



到达 DHCP 客户端需要在此网络中的地址

DHCP 客户端-服务器场景

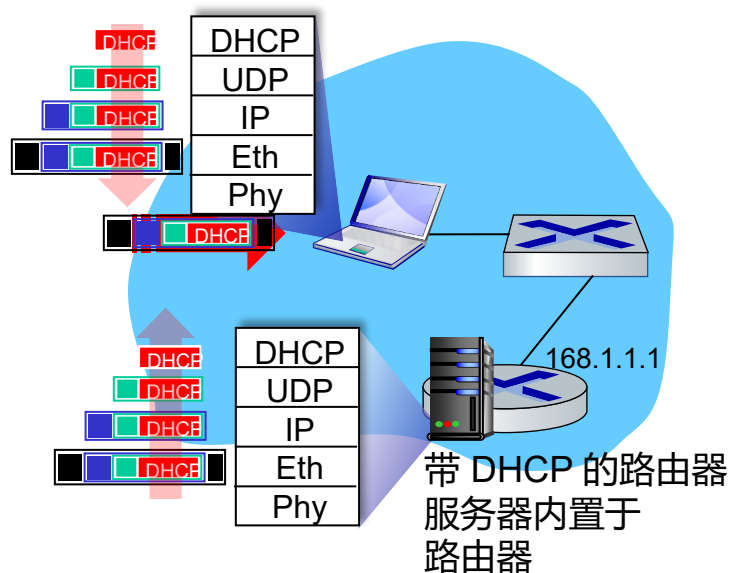


DHCP:不仅仅IP addresses

DHCP 可以返回的不仅仅是子网上分配的 IP 地址:

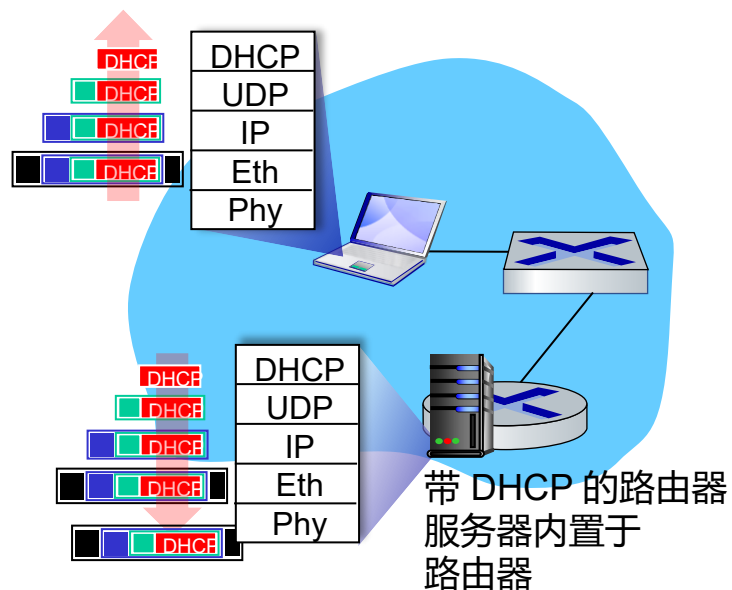
- 客户端第一跳路由器的地址
- DNS 服务器的名称和 IP 地址
- 网络掩码(表示地址的网络与主机部分)

DHCP:示例



- 连接笔记本电脑将使用 DHCP 获取 IP 地址、第一跳路由器地址、DNS 服务器地址。
- DHCP REQUEST 消息封装在 UDP 中,封装在 IP 中,封装在以太网中
- LAN 上的以太网帧广播(目的地:FFFFFFFFFFFFFF),在运行 DHCP 服务器的路由器上接收
- 以太网解复用到 IP 解复用,UDP 解复用到 DHCP

DHCP: example



- DCP 服务器制定 DHCP ACK,其中包含客户端的 IP 地址、客户端第一跳路由器的 IP 地址、DNS 服务器的名称和 IP 地址
- 封装的 DHCP 服务器回复转发给客户端,在客户端解复用到 DHCP
- 客户端现在知道其 IP 地址、DNS 服务器的名称和 IP 地址、其第一跳路由器的 IP 地址

IP 地址:如何获得?

Q:网络如何获取 IP 地址的子网部分?

A:获得分配其提供商 ISP 地址空间的一部分

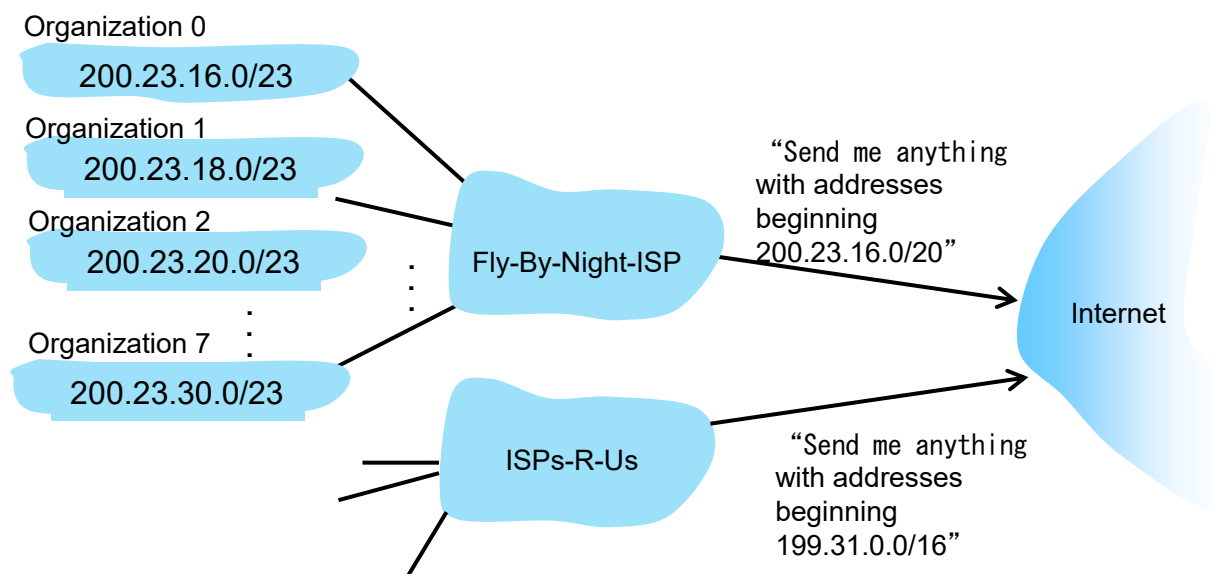
ISP's block 11001000 00010111 00010000 00000000 200.23.16.0/20

然后 ISP 可以在 8 个块中分配其地址空间:

Organization 0	<u>11001000 00010111 00010000</u>	00000000	200.23.16.0/23
Organization 1	<u>11001000 00010111 00010010</u>	00000000	200.23.18.0/23
Organization 2	<u>11001000 00010111 00010100</u>	00000000	200.23.20.0/23
...			
Organization 7	<u>11001000 00010111 00011110</u>	00000000	200.23.30.0/23

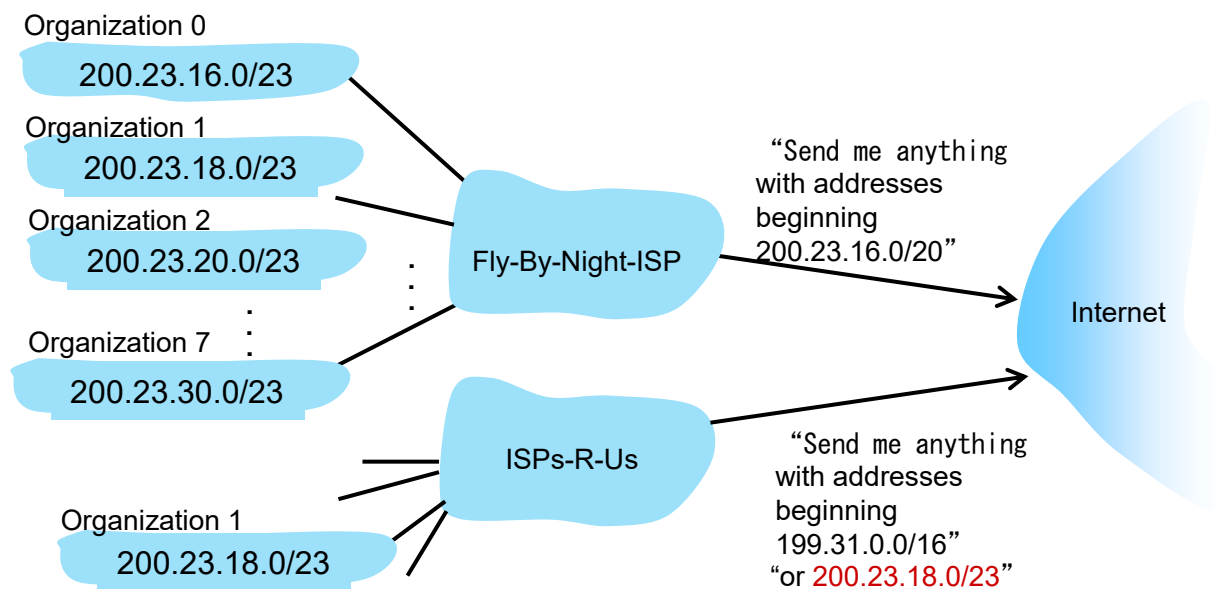
分层寻址:路由聚合

分层寻址可以有效地通告路由信息:



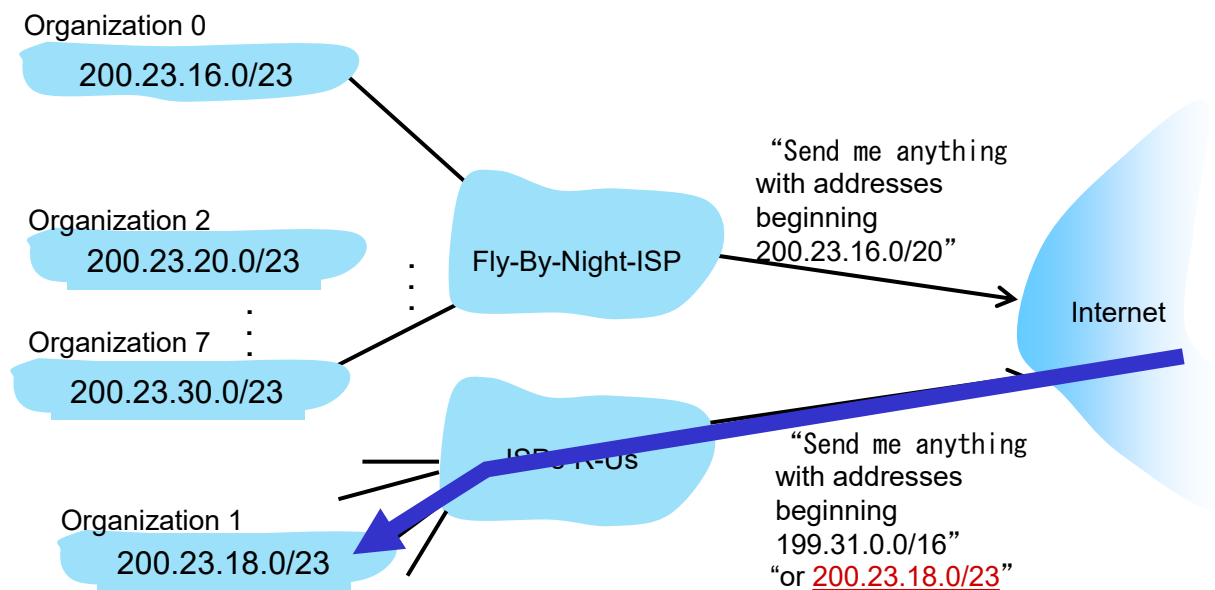
分层寻址:更具体的路线

- 组织 1 从 Fly-By-Night-ISP 迁移到 ISPs-R-Us
- ISPs-R-Us 现在通告了通往组织 1 的更具体的路线



分层寻址:更具体的路线

- 组织 1 从 Fly-By-Night-ISP 迁移到 ISPs-R-Us
- ISPs-R-Us 现在通告了通往组织 1 的更具体的路线



IP地址分配：最后的话...

Q: ISP 如何获得地址块？

A: ICANN: Internet Corporation for Assigned Names and Numbers
<http://www.icann.org/>

- 通过 5 个区域注册局 (RR) 分配 IP 地址(然后可以分配给本地注册局)
- 管理 DNS 根区,包括委托单个 TLD(.com、.edu 等)管理

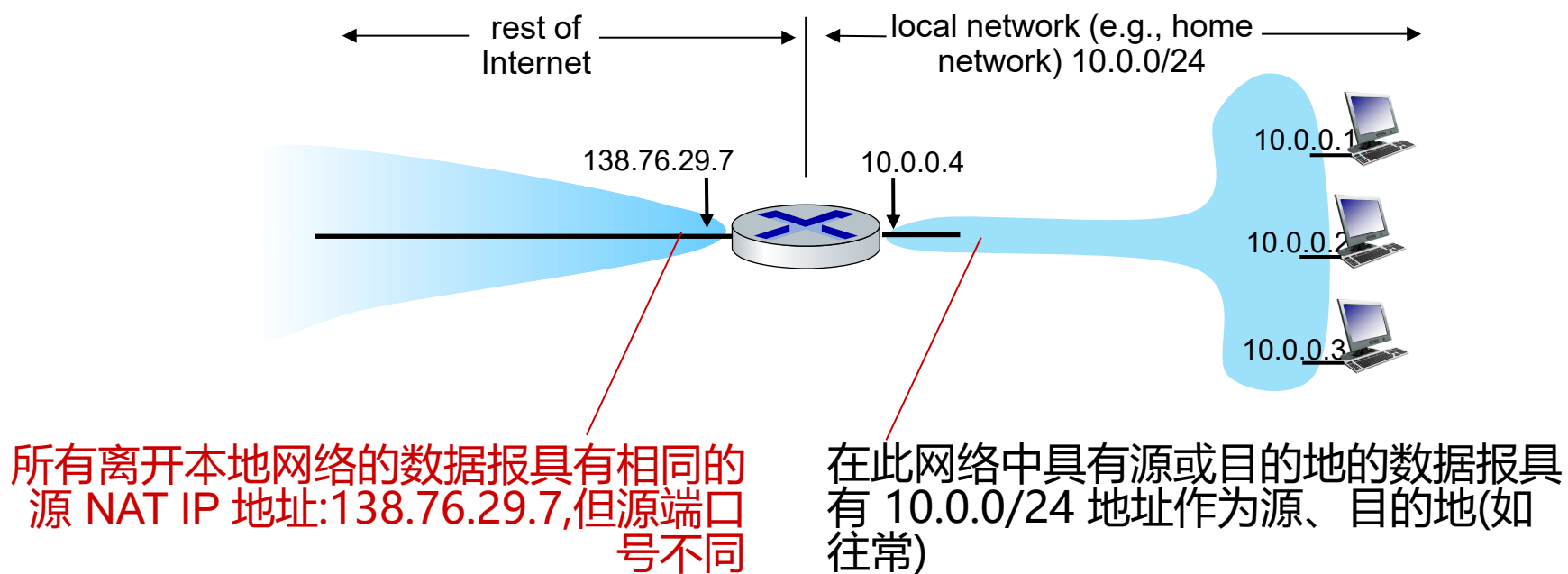
Q: 32 位 IP 地址是否足够？

- ICANN 在 2011 年将最后一块 IPv4 地址分配给 RR
- NAT (next) 帮助 IPv4 地址空间耗尽
- IPv6 有 128 位地址空间

"谁知道我们需要多少地址空间?" Vint Cerf (反思决定将 IPv4 地址设置为 32 位长)

NAT:网络地址转换

NAT:就外部世界而言,本地网络中的所有设备只共享一个 IPv4 地址



NAT:网络地址转换

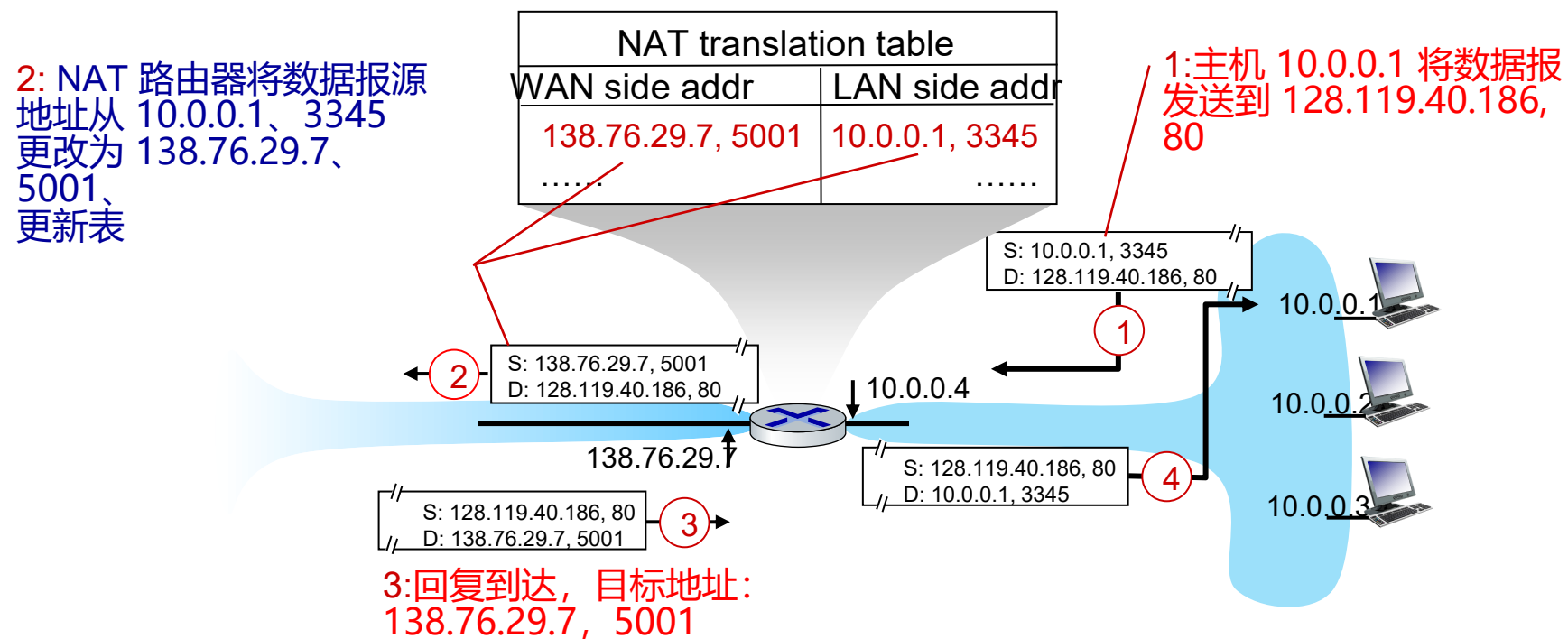
- 本地网络中的所有设备在"私有"IP 地址空间(10/8、172.16/12、192.168/16 前缀)中都有 32 位地址,只能在本地网络中使用
- 优点:
 - 所有设备只需提供商 ISP 提供一个 IP 地址
 - 可以更改本地网络中主机的地址,而不通知外界
 - 可以更改 ISP,而不更改本地网络中设备的地址
 - 安全:本地网络内的设备不可直接寻址,外界可见

NAT:网络地址转换

实施: NAT 路由器必须(透明地):

- 传出数据报:将每个传出数据报的 (源 IP 地址,端口 #) 替换为 (NAT IP 地址,新端口 #)
 - 远程客户端/服务器将使用 (NAT IP 地址,新端口 #) 作为目标地址进行响应
- 记住(在NAT翻译表中)每个(源 IP 地址,端口 #)到(NAT IP 地址,新端口 #)转换对
- 传入数据报:替换(NAT IP 地址,新端口 #),在每个传入数据报的目标字段中,相应的(源 IP 地址,端口 #)存储在 NAT 表中

NAT:网络地址转换

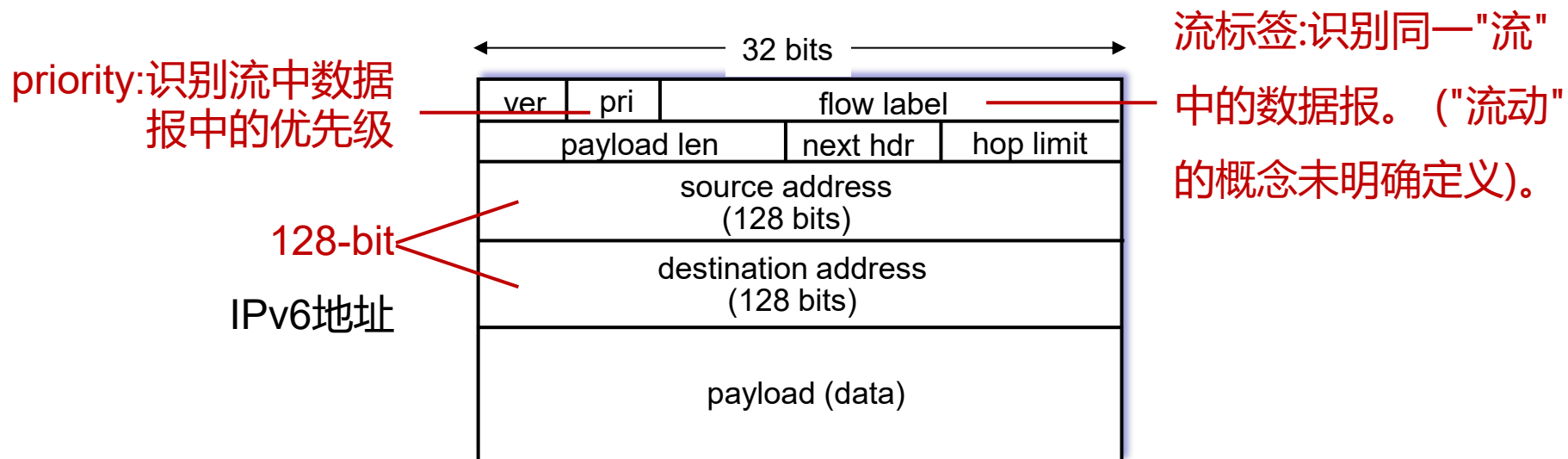


- NAT 一直有争议:
 - 路由器"应该"只处理第 3 层
 - 地址"短缺"应由 IPv6 解决
 - 违反端到端参数(端口 # 网络层设备操作)
 - NAT 遍历:如果客户端想要连接到 NAT 后面的服务器怎么办?
- 但 NAT 会留下来:
 - 广泛用于家庭和机构网络,4G/5G 蜂窝网络

IPv6:动机

- **最初的动机:** 32 位 IPv4 地址空间将完全分配
- 额外的动机:
 - 速度处理/转发:40 字节固定长度标头
 - 使"流"的不同网络层处理

IPv6 数据报格式

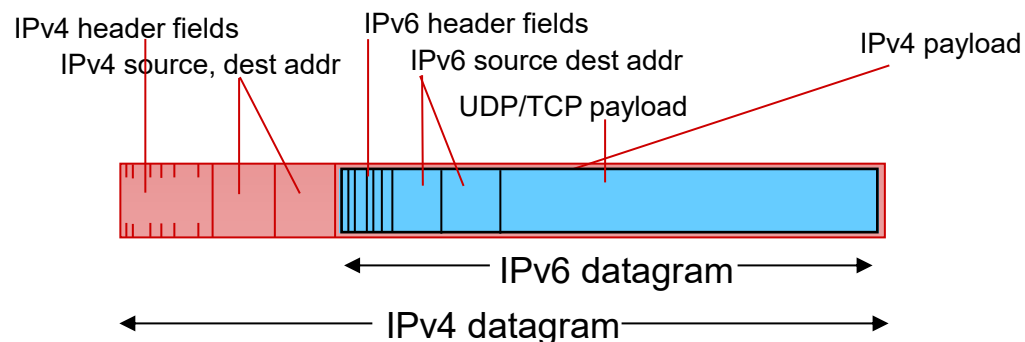


缺少什么(与 IPv4 相比):

- 无校验和(以加快路由器的处理速度)
- 无碎片/重组
- 无选项(在路由器上用作上层、下一个标头协议)

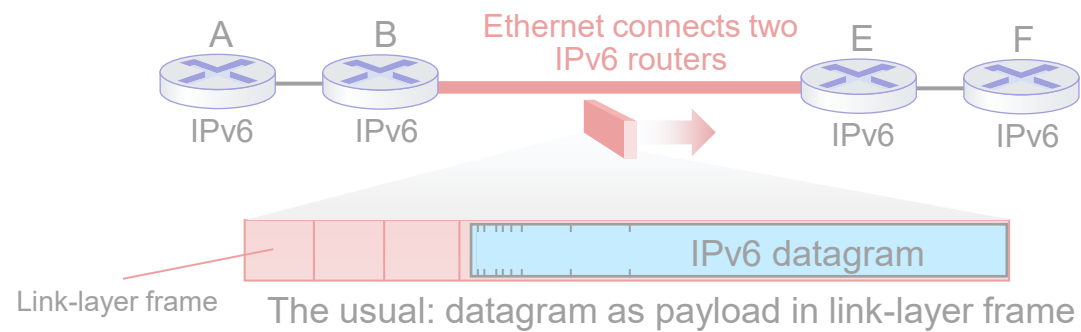
从 IPv4 过渡到 IPv6

- 并非所有路由器都可以同时升级
 - 没有“flag days”
 - IPv4 和 IPv6 混合路由器的网络将如何运行?
- **隧道:** IPv6 数据报在 IPv4 路由器之间作为负载携带 IPv4 数据报 ("包中包")
 - 隧道在其他情况下广泛使用 (4G/5G)

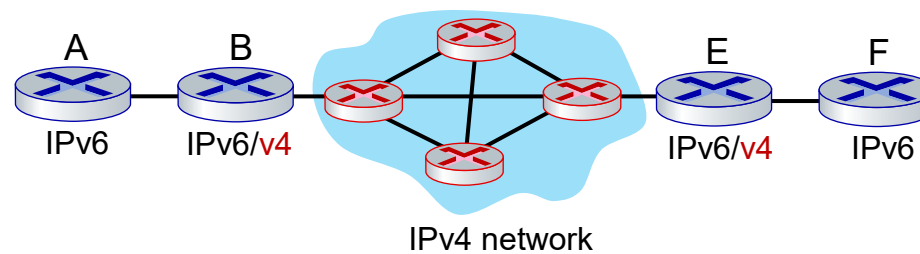


隧道和封装

以太网连接两个
IPv6 路由器:

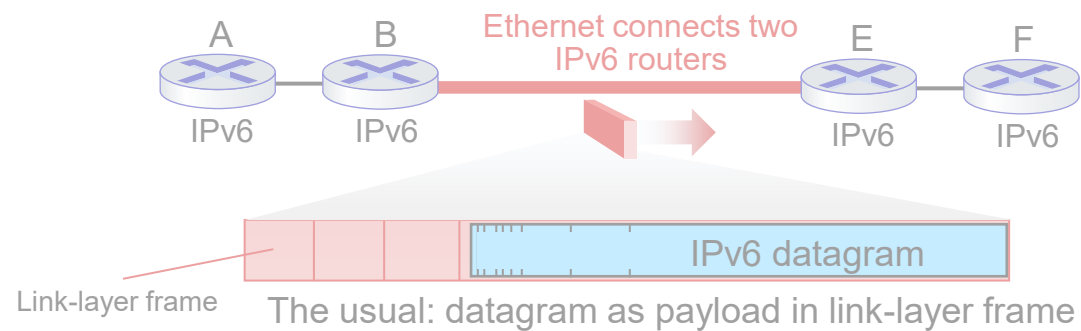


IPv4 网络连接两个
IPv6 路由器

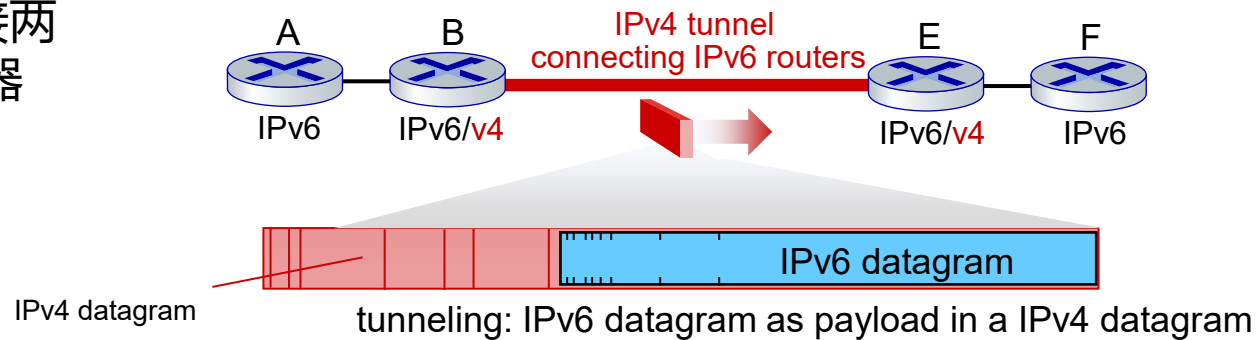


隧道和封装

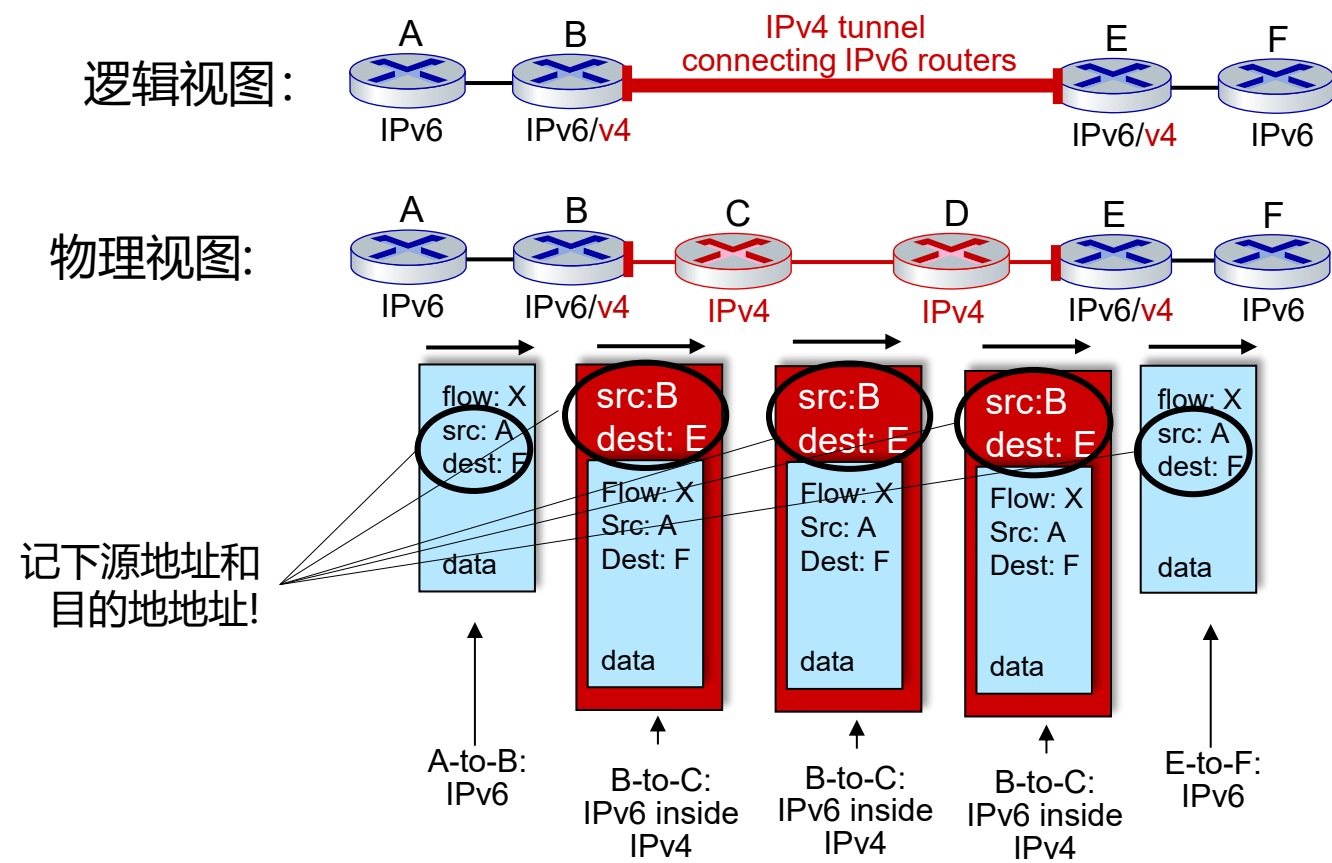
以太网连接两个
IPv6 路由器:



IPv4 隧道连接两
个 IPv6 路由器



隧道



目录

Contents

- 1 网络层与IP协议
- 2 应用层与DNS协议
- 3 Socket编程

一些网络应用程序

- 社交网络
- 网络
- 短信
- 电子邮件
- 多用户网络游戏
- 流媒体存储视频
(YouTube、Hulu、Netflix)
- P2P 文件共享
- IP 语音(例如 Skype)
- 实时视频会议
- 互联网搜索
- 远程登录
- ...

Q: 你最喜欢的?

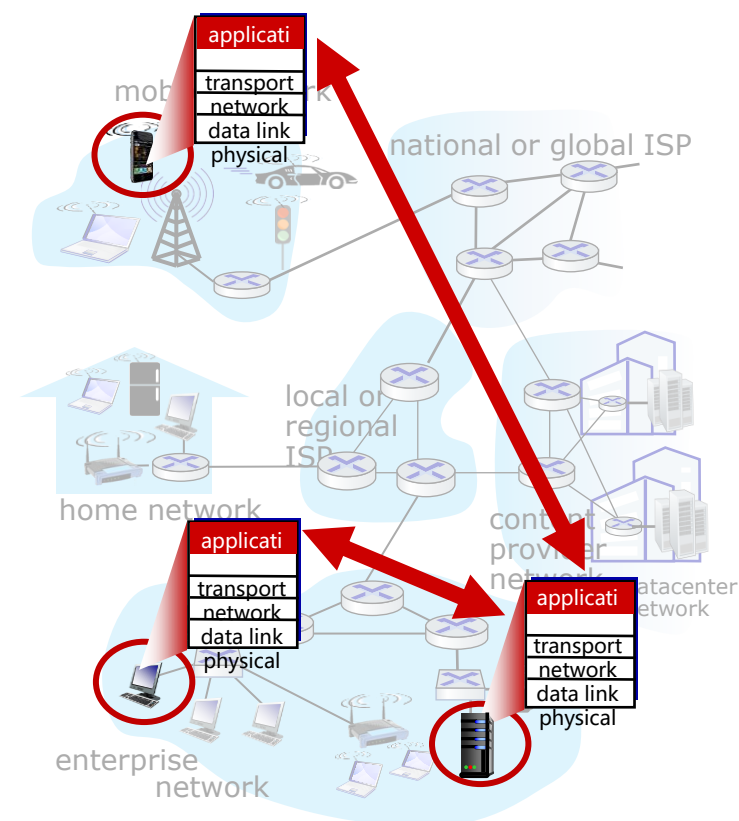
创建网络应用程序

编写程序：

- 运行在(不同的)终端系统上
- 通过网络通信
- 例如,Web 服务器软件与浏览器软件通信

无需为网络核心设备编写软件

- 网络核心设备不运行用户应用程序
- 终端系统上的应用程序允许快速开发和传播应用程序



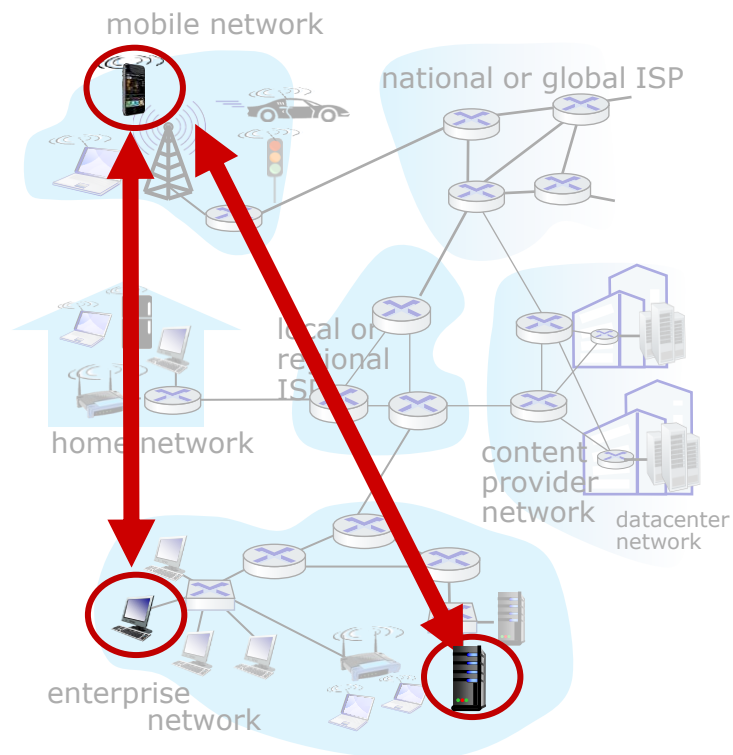
Client-server范式

server:

- 永远在线的主机
- 永久 IP 地址
- 通常在数据中心,用于扩展

clients:

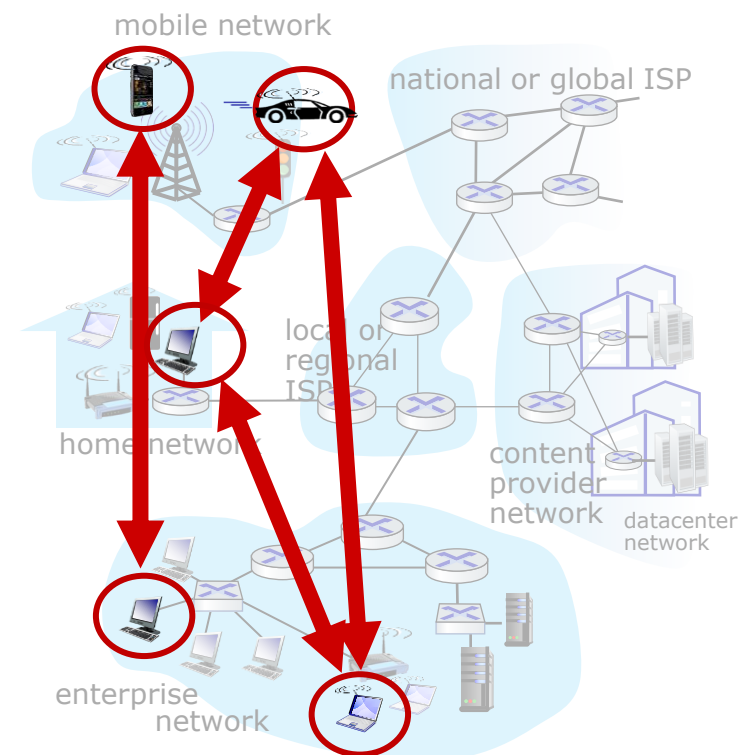
- 联系,与服务器通信
- 或断续相连
- 可能具有动态 IP 地址
- 不要直接沟通
- 例如:HTTP、IMAP、FTP



Application Layer: 2-48

Peer-peer建筑

- 没有常时服务器
- 任意终端系统直接通信
- 对等节点从其他对等节点请求服务，向其他对等节点提供服务作为回报
 - 自我扩展 - 新节点带来新的服务能力，以及新的服务需求
- 对等节点间歇性连接并更改IP地址
 - 复杂管理
- 示例：P2P文件共享



Application Layer: 2-49

process:程序在主机中运行

- 在同一主机中,两个进程使用进程间通信(由操作系统定义)进行通信
- 不同主机中的进程通过交换消息进行通信

clients, servers

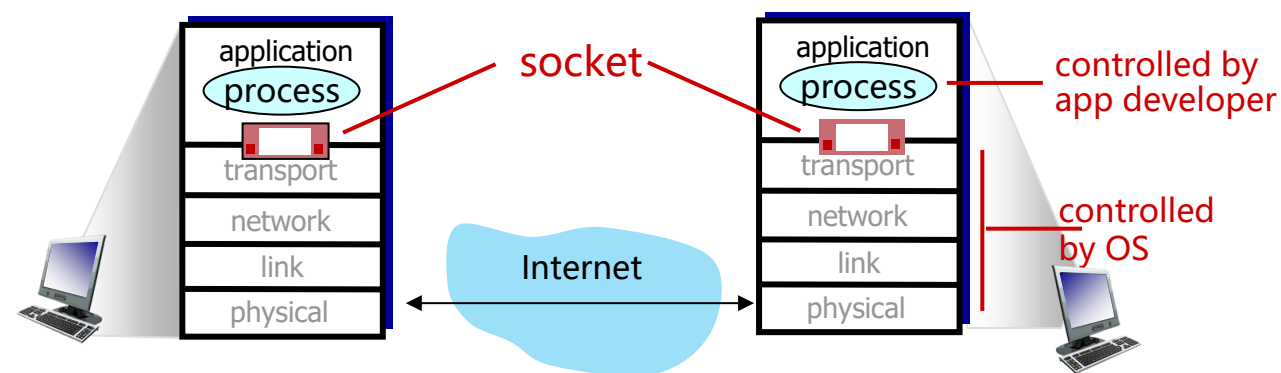
client process:发起沟通的过程

server process:等待联系的过程

- 注意:具有 P2P 架构的应用程序有客户端进程和服务端进程

Sockets

- 进程向/从其套接字发送/接收消息
- 套接字类似于门
 - 发送进程将消息推送到门外
 - 发送进程依赖于门另一侧的传输基础设施将消息传递到接收进程的套接字
 - 涉及两个套接字：每侧一个



Addressing processes

- 要接收消息,进程必须具有标识符
- 主机设备具有唯一的 32 位 IP 地址
- Q.运行进程的主机的 IP 地址足以识别进程吗?
 - A:不,许多进程可以在同一主机上运行
- *identifier 包括与主机上的进程相关的 IP 地址和端口号。*
- *端口号示例:*
 - HTTP server: 80
 - mail server: 25
- 向 Web 服务器发送 HTTP 消息 gaia.cs.umass.edu:
 - *IP address*: 128.119.245.12
 - *port number*: 80
- 更短的时间...

DNS: Domain Name System

人.许多标识符:

- SSN、姓名、护照 #

互联网主机、路由器:

- IP 地址 (32 位) - 用于寻址数据报
- "名",如 cs.umass.edu - 为人所用

Q.如何映射 IP 地址和名称,
反之亦然?

Domain Name System:

- 分布式数据库在许多层次结构中实施域名服务器
- 应用层协议:主机, 名称服务器通信以解析名称 (地址/名称转换)
 - 注:核心 Internet 功能,作为应用层协议实现
 - 网络"边缘"的复杂性

DNS服务

- 域名到IP地址转换
- 主机别名
 - 规范名, 别名
- 邮件服务器别名
- 负载分布
 - 复制的 Web 服务器:许多 IP 地址对应一个名称

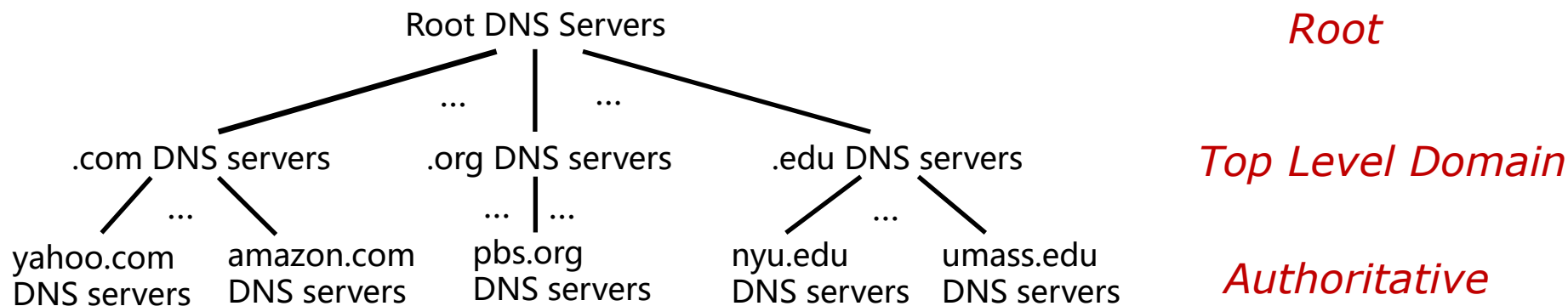
Q:为什么不集中 DNS?

- 单点故障
- 交通量
- 远程集中数据库
- 维护

A:不缩放!

仅 Comcast DNS 服务器:
每天 600B DNS 查询

DNS



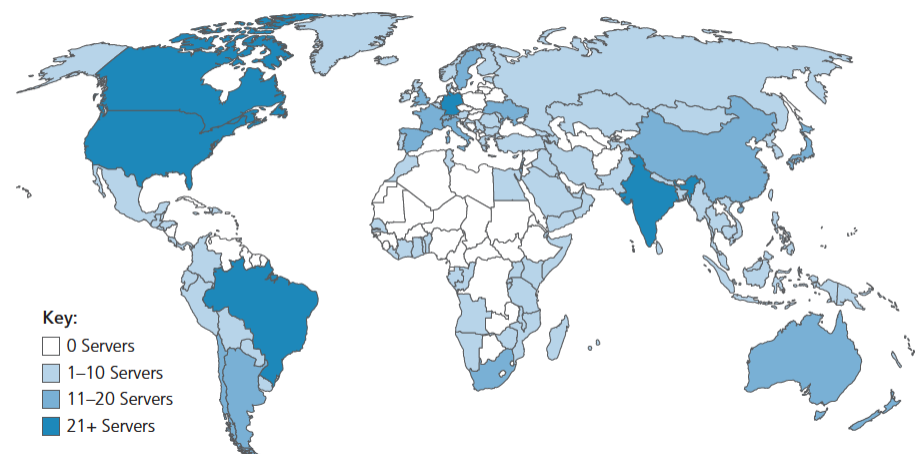
客户想要www.amazon.com的IP地址；第1个近似值:

- 客户端查询根服务器以查找 DNS 服务器.com
- 客户端.com DNS 服务器查询以获取 amazon.com DNS 服务器
- 客户端 amazon.com DNS 服务器查询以获取 IP 地址以进行
www.amazon.com

DNS: 根域名服务器

- 官方，由无法解析名称的域名服务器提供的最后手段联系方式
- 极其重要的互联网功能
 - 互联网没有它无法运行！
 - DNSSEC – 提供安全（身份验证和消息完整性）
- ICANN (Internet Corporation for Assigned Names and Numbers) 管理根 DNS 域

全球有13个逻辑根域名“服务器”，
每个“服务器”被复制多次（美国约
200个服务器）



顶级域 (TLD) 服务器:

- 负责 .com、.org、.net、.edu、.aero、.jobs、.museums 以及所有顶级国家域名,例如:.cn、.uk、.fr、.ca、.jp
- Network Solutions:.com 的权威注册表,.net TLD
- 教育原因:.edu TLD

权威 DNS 服务器:

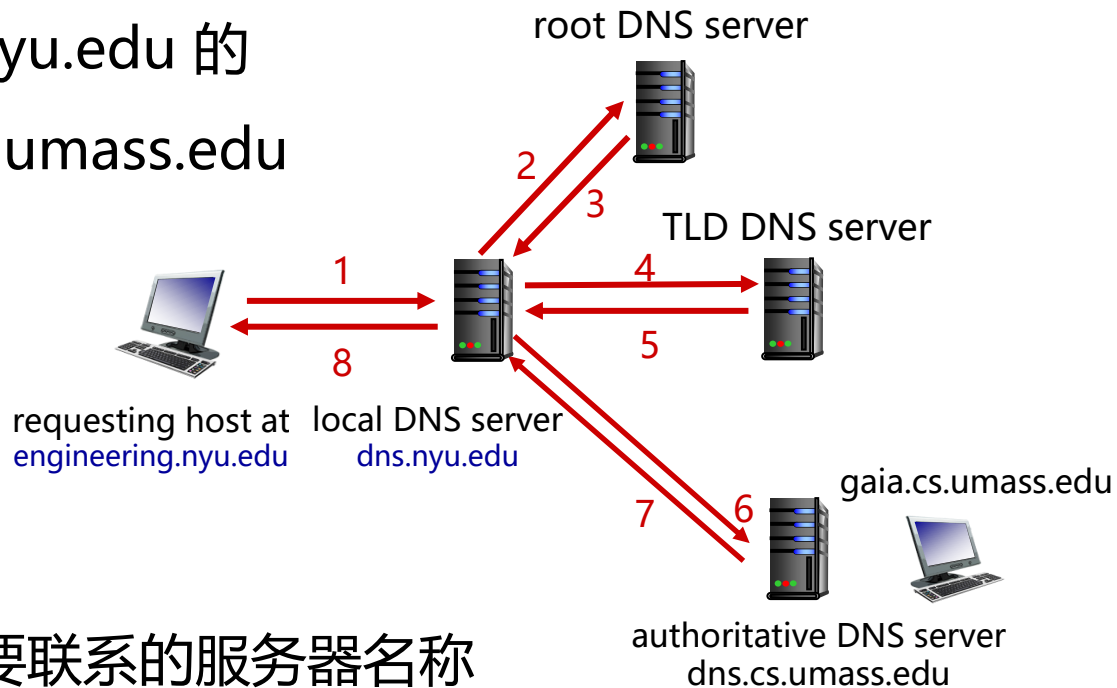
- 组织自己的 DNS 服务器,为组织的命名主机提供权威的主机名到 IP 映射
- 可以由组织或服务提供商维护

本地 DNS 名称服务器

- 不严格属于等级
- 每个 ISP(住宅 ISP、公司、大学)都有一个
- 也称为"默认名称服务器"
- 当主机进行 DNS 查询时,查询将发送到其本地 DNS 服务器
- 有最近名称到地址翻译对的本地缓存(但可能已过时!)
- 充当代理,将查询转发到层次结构中

DNS 名称解析:迭代查询

Example: engineering.nyu.edu 的主机需要 IP 地址 gaia.cs.umass.edu



迭代查询:

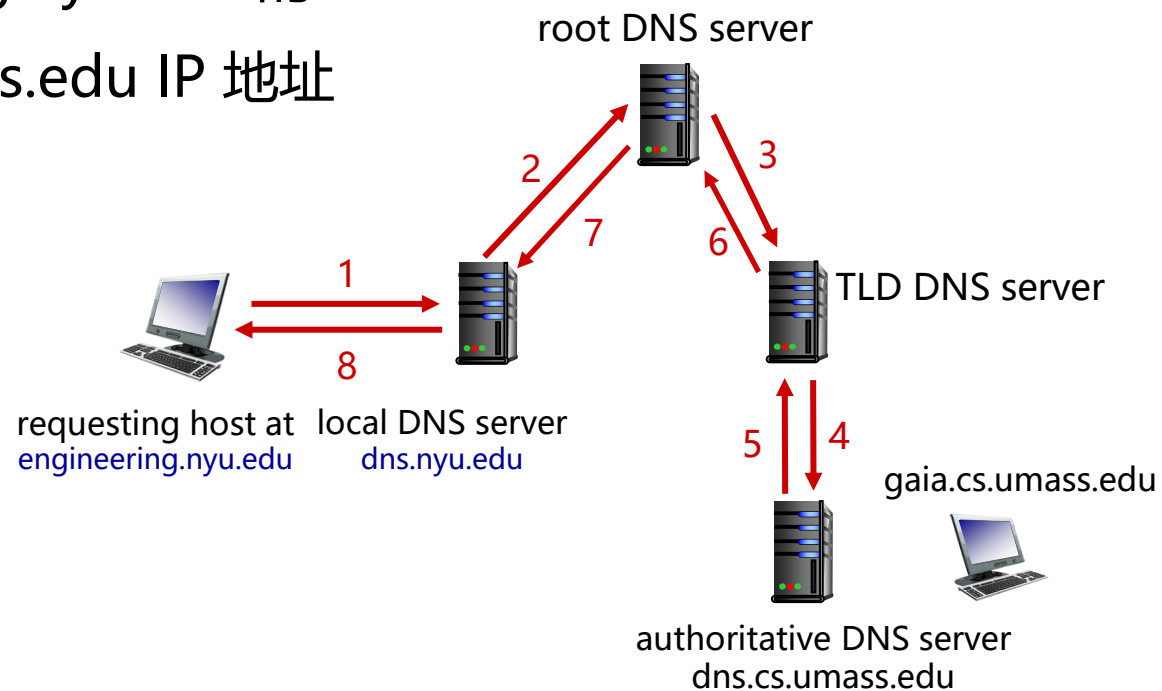
- 已联系的服务器回复要联系的服务器名称
- "我不知道这个名字,但问这个服务器"

DNS 名称解析:递归查询

Example: engineering.nyu.edu 的主机需要gaia.cs.umass.edu IP 地址

递归查询:

- 将名称解析的负担放在所联系的名称服务器上
- 上层的重担?



缓存、更新 DNS 记录

- 一旦(任何)名称服务器学习了映射,它就会缓存映射
 - 缓存条目在一段时间后超时(消失)(TTL)
 - TLD 服务器通常缓存在本地名称服务器中
 - 因此根名服务器不常访问
- 缓存的条目可能已过期(尽力翻译名称到地址!)
 - 如果名称主机更改了 IP 地址,则在所有 TTL 到期之前,可能不会在 Internet 范围内为人所知!
- 更新/通知机制建议的 IETF 标准
- RFC 2136

DNS:分布式数据库存储资源记录(RR)

RR format: (name, value, type, ttl)

type=A

- name是主机名
- value是 IP 地址

type=NS

- name是域(例如 foo.com)
- value是此域的权威名称服务器的主机名

type=CNAME

- name是某些“规范”（真实）名称的别名
- www.ibm.com 实际上是 servereast.backup2.ibm.com

type=MX

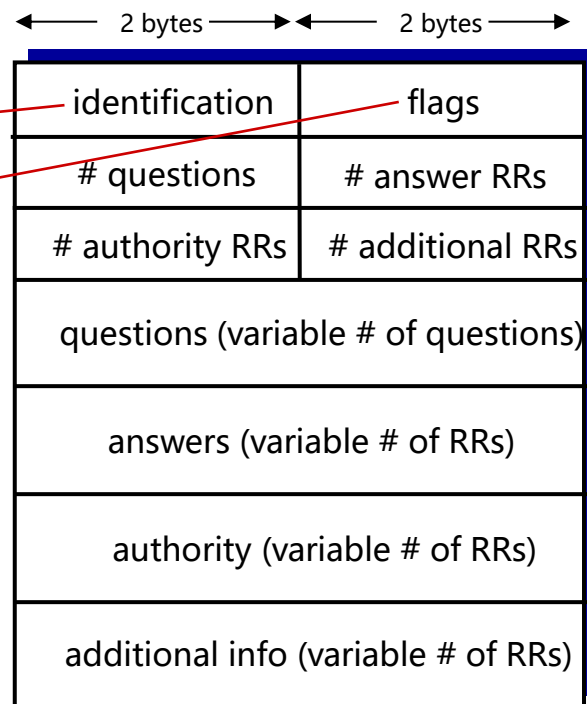
- value是与邮件服务器关联的名称name

DNS 协议消息

DNS查询和回复消息,两者格式相同:

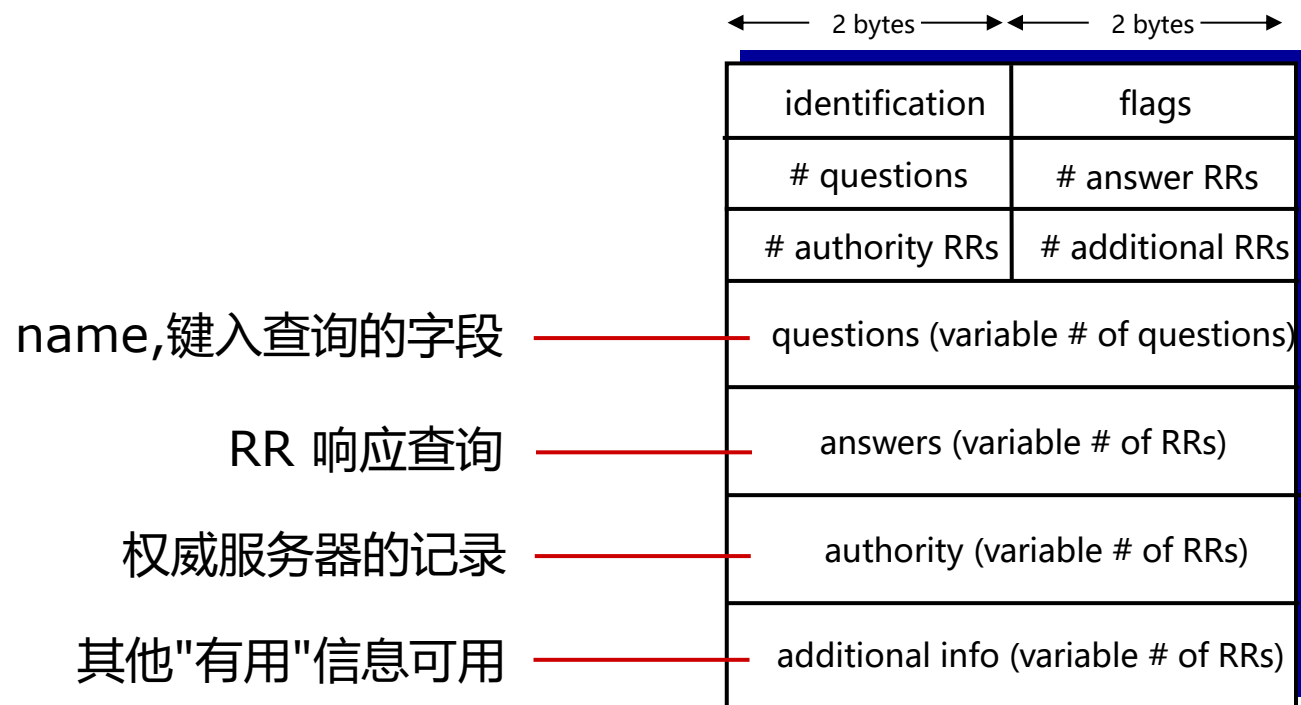
消息头:

- 识别:16 位 # 用于查询,回复
查询使用相同 #
- flags:
 - 查询或回复
 - 递归所欲
 - 递归可用
 - 回复是权威的



DNS 协议消息

DNS查询和回复消息,两者格式相同:



将记录插入 DNS

例如:新创业公司"网络乌托邦"

- 在 DNS 注册商(例如 Network Solutions)注册名称 networkutopia.com)
 - 提供权威名称服务器(主和次)的名称、IP 地址
 - registrar 将 NS、A RR 插入 .com TLD 服务器中:
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
- 使用 IP 地址212.212.212.1在本地创建权威服务器
 - www.networkutopia.com 的 A 类记录
 - networkutopia.com 的 MX 型记录

DDoS攻击

- 用流量轰炸根服务器
 - 至今不成功
 - 流量过滤
 - 本地 DNS 服务器缓存 TLD 服务器的 IP,允许绕过根服务器
- 轰炸 TLD 服务器
 - 潜在更危险

重定向攻击

- 中间人
 - 拦截 DNS 查询
- DNS 中毒
 - send bogus 依赖于 DNS 服务器,该服务器缓存

利用 DNS 进行 DDoS

- 使用欺骗源地址发送查询:目标 IP
- 需要放大

DNSSEC
C
[RFC
4033]

目录

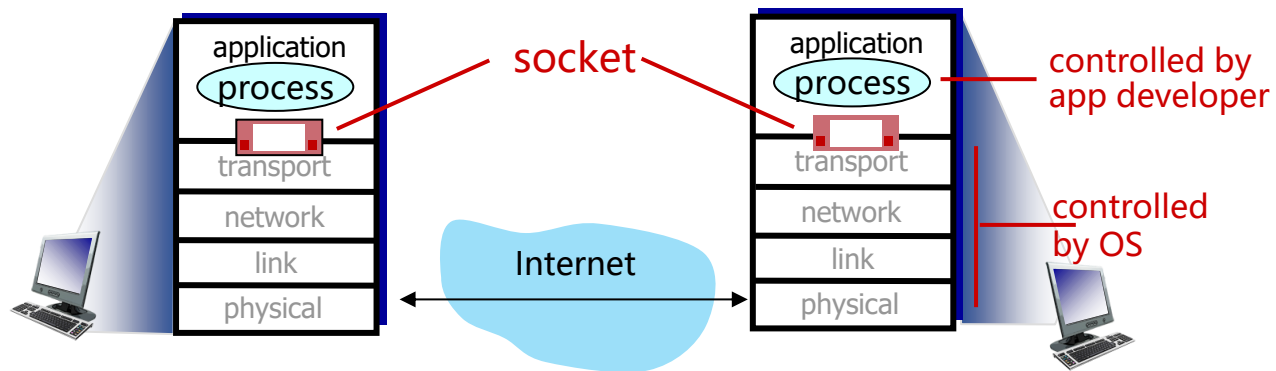
Contents

- 1 网络层与IP协议
- 2 应用层与DNS协议
- 3 **Socket编程**

Socket编程

目标:了解如何构建使用套接字进行通信的客户端/服务器应用程序

socket:申请过程与端端传输协议之间的门



两种套接字类型,用于两种传输服务:

- UDP:不可靠的数据报
- TCP:可靠,面向字节流

应用示例:

1. 客户端从其键盘上读取一行字符(数据),并将数据发送到服务器
2. 服务器接收数据并将字符转换为大写
3. 服务器将修改后的数据发送给客户端
4. 客户端接收修改后的数据,并在其屏幕上显示行

使用 UDP 进行套接字编程

UDP:客户端与服务器之间没有"连接"

- 发送数据前无需握手
- 发送者显式将 IP 目标地址和端口 # 附加到每个数据包上
- 接收者从收到的数据包中提取发送方 IP 地址和端口 #

UDP:传输的数据可能会丢失或接收失序

应用观点:

- UDP 在客户端和服务端之间提供不可靠的字节组("数据报")传输

客户端/服务器套接字交互:UDP



server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET,SOCK_DGRAM)`

↓
read datagram from
`serverSocket`

↓
write reply to
`serverSocket`
specifying
client address,
port number

client



create socket:
`clientSocket =
socket(AF_INET,SOCK_DGRAM)`

↓
Create datagram with server IP and
port=x; send datagram via
`clientSocket`

↓
read datagram from
`clientSocket`

↓
close
`clientSocket`

示例应用程序:UDP 客户端

Python UDPClient

include Python' s socket library	→	from socket import *
		serverName = 'hostname'
		serverPort = 12000
create UDP socket for server	→	clientSocket = socket(AF_INET, SOCK_DGRAM)
get user keyboard input	→	message = raw_input(' Input lowercase sentence:')
attach server name, port to message; send into socket	→	clientSocket.sendto(message.encode(), (serverName, serverPort))
read reply characters from socket into string	→	modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print out received string and close socket	→	print modifiedMessage.decode() clientSocket.close()

示例应用程序:UDP 服务器

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print ( "The server is ready to receive" )
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

create UDP socket →

bind socket to local port number 12000 →

loop forever
Read from UDP socket into message, getting client's address (client IP and port) →

send upper case string back to this client →

客户端必须联系服务器

- 服务器进程必须先运行
- 服务器必须创建了欢迎客户端联系的套接字(门)

客户端联系服务器:

- 创建 TCP 套接字,指定 IP 地址,服务器进程的端口号
- 当客户端创建套接字时:客户端 TCP 建立与服务器 TCP 的连接

- 当客户端联系时,服务器 TCP 为服务器进程创建新的套接字,以便与该特定客户端通信

- 允许服务器与多个客户端通信
- 用于区分客户端的源端口号

应用观点

TCP 提供可靠、有序的
字节流传输("管道")
客户端与服务器之间

客户端/服务器套接字交互:TCP



server (running on hostid)

client



create socket,
port= x , for
incoming request:
`serverSocket = socket()`

wait for incoming
connection request
`connectionSocket = serverSocket.accept()`

read request from
`connectionSocket`

write reply to
`connectionSocket`

close
`connectionSocket`

create socket,
connect to `hostid`, port= x
`clientSocket = socket()`

send request using
`clientSocket`

read reply from
`clientSocket`

close
`clientSocket`

TCP
connection setup

示例应用程序:TCP 客户端

Python TCPClient

create TCP socket for
server, remote port 12000

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = raw_input( 'Input lowercase sentence:' )
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ( 'From Server:' , modifiedSentence.decode())
clientSocket.close()
```

No need to attach server name,
port

示例应用程序:TCP 服务器

Python TCPServer

create TCP welcoming socket	→	from socket import *
		serverPort = 12000
server begins listening for incoming TCP requests	→	serverSocket = socket(AF_INET,SOCK_STREAM)
		serverSocket.bind((" ",serverPort))
		serverSocket.listen(1)
		print 'The server is ready to receive'
loop forever	→	while True:
server waits on accept() for incoming requests, new socket created on return	→	connectionSocket, addr = serverSocket.accept()
read bytes from socket (but not address as in UDP)	→	sentence = connectionSocket.recv(1024).decode()
		capitalizedSentence = sentence.upper()
		connectionSocket.send(capitalizedSentence.encode())
close connection to this client (but not welcoming socket)	→	connectionSocket.close()